

“Ultima Corsa”

VERSIONE FINALE – Le modifiche sono riportate in rosso.

Progettare e implementare un'applicazione web di un gioco single player ispirato al gioco da tavolo “Race the Rails”.

Nel gioco, al giocatore vengono assegnati una stazione di partenza e di arrivo che varia a ogni partita all'interno di una rete metropolitana fittizia. Il giocatore deve pianificare ed eseguire un percorso valido prima che il tempo scada, raccogliendo o perdendo monete lungo il tragitto a causa di eventi casuali. L'obiettivo è arrivare a destinazione con il punteggio più alto possibile.

Il gioco si basa su una **rete metropolitana** composta da un insieme fisso di stazioni collegate da linee di metropolitana. Sia le stazioni, sia le linee, hanno un nome univoco. Un esempio semplificato della rete potrebbe essere:

- Linea Rossa: Centrale <-> Porta Velaria <-> Crocevia del Falco <-> Piazza delle Lanterne
- Linea Blu: Centrale <-> Fontana Oscura <-> Borgo Sereno <-> Viale dei Mosaici
- Linea Verde: Porta Velaria <-> Fontana Oscura <-> Torre Cinerea <-> Campo dell'Eco
- Linea Gialla: Piazza delle Lanterne <-> Torre Cinerea <-> Viale dei Mosaici <-> Campo dell'Eco

I nomi delle stazioni e delle linee nonché i collegamenti tra le stazioni sono lasciati allo studente/ssa. La rete metropolitana deve avere *almeno 4 linee, almeno 12 stazioni e almeno 3 stazioni di interscambio* (cioè stazioni servite da più di una linea). **Le stazioni di interscambio non possono essere più della metà delle stazioni (quindi se ci sono 12 stazioni, al massimo la rete può avere 6 stazioni di interscambio).** La rete non cambia all'interno del gioco. Su base probabilistica, possono capitare *almeno 8 eventi diversi* durante una tratta; ogni evento è composto da una descrizione e da un effetto (intero positivo o negativo, da -4 a +4). Gli studenti e le studentesse possono definire gli eventi a loro piacere. Esempi di evento possono includere: “Viaggio tranquillo, 0 monete”, “Binario sbagliato, -2 monete”, “Passeggero gentile, +1 moneta”.

La rete e gli eventi devono essere memorizzati nel server e recuperati dal client nei momenti opportuni.

L'applicazione permette di giocare più partite. Ogni partita inizia con 20 monete e si articola nelle seguenti fasi:

1. **Setup.** Il giocatore vede la mappa della rete con tutte le stazioni, le loro connessioni e le linee. Quando il giocatore è pronto a giocare, passa alla fase successiva.
2. **Pianificazione.** Il giocatore vede tre elementi nella pagina:
 - a. la mappa della rete, che mostra solo le stazioni con i rispettivi nomi ma senza le linee che le collegano;
 - b. l'elenco di tutte le tratte (coppie di stazioni collegate, ad esempio, Porta Velaria—Fontana Oscura);
 - c. una stazione di partenza e una di arrivo, assegnate casualmente dal server, dove la stazione di arrivo deve essere raggiungibile nella rete dalla stazione di partenza con una distanza minima di almeno 3 **tratte** tra di loro (**ad esempio, Centrale -> Porta Velaria -> Crocevia del Falco -> Piazza delle Lanterne sono 3 tratte**).

Da quando inizia questa fase, il giocatore ha *90 secondi* per scorrere l'elenco delle tratte, ricostruire mentalmente la rete e costruire il proprio percorso selezionando le tratte in sequenza. **Ogni tratta può essere selezionata dall'elenco una sola volta durante una partita.** Il percorso deve partire dalla stazione di partenza e terminare alla stazione di arrivo assegnate. Prima della scadenza dei 90 secondi, il giocatore deve inviare il percorso costruito. Se il tempo scade, la fase di pianificazione termina automaticamente con il percorso costruito fino a quel momento (anche se incompleto o non valido). Un percorso è valido quando inizia e termina nelle stazioni assegnate e ogni tratta è raggiungibile con una delle linee, con i cambi di linea possibili solo nelle stazioni di interscambio. **Un percorso può contenere la stessa stazione più di una volta se compare in tratte diverse.**

3. **Esecuzione.** L'applicazione web valida il percorso inviato e, per ogni tratta del viaggio (ogni tappa da una stazione a quella successiva), seleziona casualmente un evento tra quelli disponibili e il suo effetto viene applicato al totale di monete della partita. L'applicazione web mostra le tappe una alla volta, in sequenza, visualizzando l'evento che è capitato e il totale aggiornato delle monete. Se il percorso inviato non è valido o è incompleto, questa fase viene saltata e il giocatore perde tutte le 20 monete della partita (ottenendo quindi un punteggio di zero monete).
4. **Risultato.** Al giocatore viene mostrato il punteggio finale, che corrisponde alle monete rimanenti nella partita. Se il punteggio finale risultasse negativo, verrà memorizzato e mostrato come zero. Il giocatore può scegliere se cominciare una nuova partita o meno.

Gli **utenti registrati** (cioè, dopo aver fatto login) possono giocare tutte le partite che vogliono. Inoltre, il miglior risultato delle loro partite deve essere visualizzato in una classifica generale, disponibile in una pagina dedicata dell'applicazione.

Gli **utenti anonimi** (cioè, i visitatori del sito), invece, possono solo visualizzare le istruzioni di gioco (senza vedere la mappa della rete). Non avranno accesso a nessuna funzionalità degli utenti registrati.

L'organizzazione di queste specifiche in diverse schermate (e possibilmente su diverse route) è lasciata allo studente.

Requisiti del progetto

- L'architettura dell'applicazione e il codice sorgente devono essere sviluppati adottando le migliori pratiche (best practice) di sviluppo del software, in particolare per le single-page application (SPA) che usano React e HTTP API. Le API devono essere protette con cura e il front-end non dovrebbe ricevere informazioni non necessarie.
- L'applicazione deve essere pensata per un browser desktop. La responsività per dispositivi mobile non è richiesta né valutata.
- Il progetto deve essere realizzato come applicazione React, che interagisce con API HTTP implementate in Node.js+Express. La versione di Node.js deve essere quella usata durante il corso (24.x, LTS). Il database deve essere memorizzato in un file SQLite. Il linguaggio di programmazione deve essere JavaScript.
- La comunicazione tra il client ed il server deve seguire il pattern dei "due server", configurando correttamente CORS e con React in modalità "development" con lo Strict Mode attivato.
- La valutazione del progetto sarà effettuata navigando all'interno dell'applicazione. Non saranno testati né usati il bottone di "refresh" né l'impostazione manuale di un URL (tranne

/), e il loro comportamento non è specificato. Inoltre, l'applicazione non dovrà mai "autoricararsi" come conseguenza dell'uso normale dell'applicazione stessa.

- La directory radice del progetto deve contenere un file README.md e contenere due subdirectories (client e server). Il progetto deve poter essere lanciato con i comandi: "cd server; nodemon index.js" e "cd client; npm run dev". Un template con lo scheletro delle directory del progetto è disponibile nel repository dell'esame. Si può assumere che nodemon sia già installato a livello di sistema. Nessun altro modulo sarà disponibile globalmente.
- L'intero progetto deve essere consegnato tramite GitHub, nel repository creato da GitHub Classroom.
- Il progetto **non deve includere** le directory node_modules. Esse devono essere ricreabili tramite il comando "npm install" subito dopo "git clone".
- Il progetto può usare librerie popolari e comunemente adottate (per esempio, day.js, react-bootstrap, ecc.), se applicabili e utili. Tali librerie devono essere correttamente dichiarate nei file package.json cosicché il comando npm install le possa scaricare ed installare.
- L'autenticazione dell'utente (login e logout) e l'accesso alle API devono essere realizzati tramite Passport.js e cookie di sessione. Le credenziali devono essere memorizzate in formato hashed e con sale. La registrazione di un nuovo utente non è richiesta né valutata.

Requisiti di qualità

In aggiunta all'implementazione delle funzionalità richieste dell'applicazione, saranno valutati i seguenti requisiti di qualità:

- Progettazione e organizzazione del database.
- Progettazione delle HTTP API.
- Organizzazione dei componenti React e delle route.
- Uso corretto dei pattern di React (comportamento funzionale, hook, stato, contesto ed effetti). Questo include evitare la manipolazione diretta del DOM.
- Chiarezza del codice.
- Assenza di errori (e warning) nella console del browser (tranne quelli causati da errori nelle librerie importate).
- Assenza di crash dell'applicazione o eccezioni non gestite.
- Validazione essenziale dei dati (in Express e in React).
- Usabilità e facilità d'uso base.
- Originalità della soluzione.

Requisiti del database

- Il database del progetto deve essere realizzato dallo studente e deve essere precaricato con almeno 4 linee, almeno 12 stazioni, almeno 3 stazioni di interscambio, almeno 8 eventi diversi e almeno 3 utenti registrati. Due degli utenti registrati devono aver giocato alcune partite con successo.

Contenuto del file README.md

Il file README.md deve contenere le seguenti informazioni (un template è disponibile nel repository del progetto). In genere, ogni spiegazione non dovrebbe essere più lunga di 2-3 righe.

1. Server-side:

- a. Una lista delle API HTTP offerte dal server, con una breve descrizione dei parametri e degli oggetti scambiati.
 - b. Una lista delle tabelle del database, con il loro scopo.
2. Client-side:
 - a. Una lista delle route dell'applicazione React, con una breve descrizione dello scopo di ogni route.
 - b. Una lista dei principali componenti React implementati nel progetto.
3. In generale:
 - a. Due screenshot dell'applicazione, **uno con la classifica generale e uno durante una partita**. Le immagini vanno embeddate nel README linkando due immagini da inserire nel repository stesso.
 - b. Username e password degli utenti registrati.
 - c. Uso di tool di AI durante lo svolgimento del progetto, indicando per quale scopo e come è stato verificato o adattato il loro output. Se non è stato usato nessun tool, indicarlo.

Procedura di consegna

Per sottomettere correttamente il progetto è necessario:

- Essere **iscritti** all'appello.
- Usare il **link** fornito per **unirsi alla classroom** di questo appello su GitHub Classroom (cioè, correttamente **associare** il proprio nome utente GitHub con la propria matricola studente) e **accettare l'assignment**.
- Fare il **push del progetto** nel **branch main** del repository che GitHub Classroom ha generato per ognuno. L'ultimo commit (quello che si vuole venga valutato) deve essere **taggato** con il tag **final** (nota: final deve essere scritto tutto minuscolo e senza spazi ed è un 'tag' git, non un 'messaggio di commit').

Nota: per taggare un commit, si possono usare (dal terminale) i seguenti comandi:

```
# ensure the latest version is committed
git commit -m "...comment..."
git push

# add the 'final' tag and push it
git tag final
git push origin --tags
```

In alternativa, è possibile inserire un tag dall'interfaccia web di GitHub (seguire il link 'Create a new release').

Per testare la propria sottomissione, questi sono gli esatti comandi che saranno usati per scaricare ed eseguire il progetto. Potreste volerli provare in una directory vuota:

```
git clone ...yourCloneURL...
cd ...yourProjectDir...
git pull origin main # just in case the default branch is not main
git checkout -b evaluation final # check out the version tagged with
'final' and create a new branch 'evaluation'
(cd server ; npm install; nodemon index.js)
(cd client ; npm install; npm run dev)
```

Assicurarsi che tutti i pacchetti (package) necessari siano scaricati tramite i comandi `npm install`. Fate attenzione: se alcuni pacchetti sono stati installati a livello globale, potrebbero non apparire come dipendenze necessarie. Controllare sempre con un'installazione pulita.

Fate attenzione al fatto che Linux è case-sensitive nei nomi dei file, mentre macOS e Windows non lo sono. Pertanto, si controllino con particolare cura le maiuscole/minuscole usate nei nomi dei file e negli `import`.